RMQ 是 **Range Minimum Query** 的缩写，中文意为 **“区间最值查询”**。

在计算机科学（特别是算法竞赛和数据结构领域）中，RMQ 问题是指：对于一个给定的数组 A，如何在多次查询中，快速找到数组在指定区间 $[L, R]$ 内的最小值（或最大值）。

1. 核心问题定义

- **输入**：一个长度为 n 的序列 A，以及若干组查询 (L, R)。
- **输出**：对于每组查询，返回 $\min(A[L], A[L+1], \ldots, A[R])$。

2. 常见的解决方案

根据数据的特点（是否会发生修改）和查询次数的要求，通常有以下几种解决方法：

① 暴力搜索 (Naive Approach)

- **做法**：每次查询都遍历一遍区间 $[L, R]$。
- **时间复杂度**：预处理 $O(1)$，单次查询 $O(n)$。
- **适用场景**：查询次数极少。

② 稀疏表 (Sparse Table - ST表) —— 最经典解法

- **做法**：利用**倍增（Dynamic Programming + Binary Lifting）**思想。预先计算出长度为 2^j 的区间的最小值。
- **时间复杂度**：预处理 $O(n \log n)$，单次查询 $O(1)$。
- **优点**：查询速度极快（常数级）。
- **缺点**：**只适用于静态数据**。如果数组中的值会发生修改，ST表需要重新计算，代价太大。

③ 线段树 (Segment Tree)

- **做法**：将数组构建成一棵树状结构，每个节点存储该区间的最小值。
- **时间复杂度**：预处理 $O(n)$，单次查询 $O(\log n)$，单次修改 $O(\log n)$。
- **优点**：**支持动态修改**。如果数组元素会变动，线段树是首选。
- **缺点**：常数比 ST 表大，代码实现相对复杂。

④ 树状数组 (Fenwick Tree)

- **做法**：虽然树状数组主要用于区间求和，但经过特殊修改也可以处理 RMQ。
- **时间复杂度**：预处理 $O(n \log n)$，查询和修改均为 $O(\log n)$。

- **缺点**：处理 RMQ 不如线段树直观，通常用于特定限制条件的场景。

3. 不同算法对比总结

算法	预处理复杂度	查询复杂度	是否支持修改	适用场景
暴力	$O(1)$	$O(n)$	是	数据量极小
ST 表	$O(n \log n)$	$O(1)$	否	**静态数据，大量查询**
线段树	$O(n)$	$O(\log n)$	**是**	**动态修改数据**
笛卡尔树	$O(n)$	$O(1)$	否	理论最优，常用于高级算法

4. RMQ 的重要应用

RMQ 问题不仅仅是为了找最大最小值，它在很多高级算法中是核心组件：

1. **LCA (最近公共祖先)**：通过将树进行欧拉序列化（DFS 序），可以将 LCA 问题转化为 RMQ 问题，从而实现 $O(1)$ 的查询。
2. **后缀数组 (Suffix Array)**：在处理字符串匹配时，求两个后缀的最长公共前缀 (LCP)，本质上是在 LCP 数组上进行 RMQ 查询。
3. **计算几何**：在一些复杂的几何算法中，用于处理坐标区间的极值。

总结

- 如果你处理的是**不会变**的数组，且查询次数很多，请用 **ST 表**。
- 如果你处理的是**会发生改变**的数组，请用 **线段树**。